COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CpE21S2
DATE PERFORMED:	9/16/2026	**DATE SUBMITTED:**	9/17/2026
NAME:	Fernandez, Cedric S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 9.1 **Implementing Trees**

1. PROCEDURE OUTPUT

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

int main() {

    // Creating the root node
    BinaryTree<char>* root = new BinaryTree<char>('A');

    // Creating children of root node A
    root->insertLeftChild('B');
    root->insertRightChild('C');

    // Insert children to node B
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertRightChild('F');

    // Insert children to node C
    root->getRightChild()->insertLeftChild('G');
    root->getRightChild()->insertRightChild('H');

    // Print the root node
    std::cout << "Root: " << root->getRootVal() << "\n";

    // Children of the root node
    std::cout << "Root left child: "
              << root->getLeftChild()->getRootVal() << std::endl;

    std::cout << "Root right child: "
              << root->getRightChild()->getRootVal() << std::endl;

    // Children of subtree B
    std::cout << "Node B left child: "
              << root->getLeftChild()->getLeftChild()->getRootVal()
              << std::endl;

    std::cout << "Node B right child: "
              << root->getLeftChild()->getRightChild()->getRootVal()
              << std::endl;

    // Change node B's right child from F to E
    root->getLeftChild()->getRightChild()->setRootVal('E');

    std::cout << "Node B right child after changing value: "
              << root->getLeftChild()->getRightChild()->getRootVal()
              << std::endl;

    return 0;
}
```

OBSERVATION

For the following code we make us of the header file binaryTree.h in order to make a binary tree starting with the root A then attached to it is B and C acting as a left and right child, then attaching to B and C are D and F under B and G and H under C also acting as a child then at the last part is the setRootVal() which was used to overwrite or replace F with E so that the following node will store F instead of E

binaryTree.h

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree {
private:
    T key;
    //pointers:
    BinaryTree<T>* leftChild;
    BinaryTree<T>* rightChild;
public:

    //constructor:
    BinaryTree(T rootObj);

    //adding/inserting a child node:
    void insertLeftChild(T newNode);
    void insertRightChild(T newNode);

    BinaryTree<T>* getLeftChild();
    BinaryTree<T>* getRightChild();

    T getRootVal();
    void setRootVal(T obj);

    bool isLeaf();

};

template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

template <typename T>
void BinaryTree<T>::insertLeftChild(T newNode){

    if(leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }else{
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T> (newNode);
        //move the existing left child below the new Node
        newChild->leftChild = leftChild;
        leftChild = newChild;
    }
}

template <typename T>
void BinaryTree<T>::insertRightChild(T newNode) {
    if(rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    }
    else{
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

```

```cpp

template<typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template<typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}

template<typename T>
void BinaryTree<T>::setRootVal(T obj){
    key = obj;
}

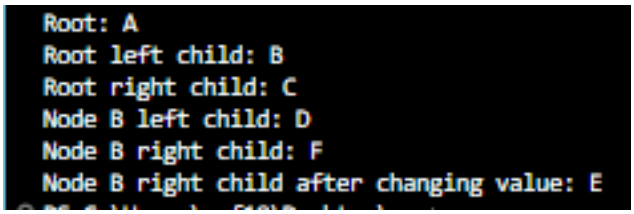
template<typename T>
bool BinaryTree<T>::isLeaf(){

    return leftChild == nullptr && rightChild == nullptr;
}
#endif
```

OBSERVATION

For the header file we created a class called BinaryTree which is where we will have each node have a max capacity of 2 children one called left child and the other called right child then with the use of a constructor we can make it so that the value inputted is initialized. While also making use of the general insertion method to add children to the tree and the getter methods to give us access to the payload stored. Lastly the isLeaf is the one that checks if a node has no children

2. OUTPUT



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n/a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n/a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n/a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.